

1 作业内容

- ①下载并安装PyTorch和Transformers
- ②使用Transformers库实现一个模型(任意)进行任意数量轮次的对话(最好超过两轮)，并保留历史记录;
- ③熟悉分词，并分别将一个句子(任何)转换为其分词形式和ID形式。
- ④熟悉模型的结构，并打印该模型任意层的一个(或多个)attention层的输出。

2 代码实现

2.1 多轮对话

目前下载好的PyTorch版本为2.5.1+cu121，Transformers版本4.52.3，CUDA版本 12.1 (nvcc 12.1.66)

此电脑 > OS (C:) > 用户 > Lenovo > .cache > huggingface > hub

名称	修改日期	类型
.locks	2025/10/8 11:33	文件夹
models--bert-base-chinese	2025/5/15 19:01	文件夹
models--distilbert--distilbert-base-u...	2025/10/8 0:14	文件夹
models--gpt2	2025/10/7 21:25	文件夹
models--Qwen--Qwen3-0.6B	2025/10/8 11:40	文件夹
models--TinyLlama--TinyLlama-1.1B-...	2025/10/7 23:18	文件夹

选择用Qwen/Qwen3-0.6B模型，轻量且效果好。

```
from transformers import AutoTokenizer, AutoModelForCausalLM
model_name="Qwen/Qwen3-0.6B"
#导入模型
tokenizer=AutoTokenizer.from_pretrained(model_name)
model=AutoModelForCausalLM.from_pretrained(model_name)

chat_history = []#全局变量记录历史记录
```

如果要实现带历史记录的输入，可以用tokenizer.apply_chat_template方法，可以将对话数据格式化为可接受的输入格式，支持Qwen的对话模板，能够自动处理历史记录。

chat_history以**列表嵌套字典**的格式存储历史记录，tokenizer.apply_chat_template可以自动将其转化为对话格式。

定义一个函数来调用模型进行对话。函数输入为用户新的语句，模型输入为整个的历史记录，输出为模型的最后一次输出。

```
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

model_name="Qwen/Qwen3-0.6B"

tokenizer=AutoTokenizer.from_pretrained(model_name)
model=AutoModelForCausalLM.from_pretrained(model_name)

chat_history=[]

def chat(user_input,max_new_tokens=256):
    global chat_history
```

```

chat_history.append({"role": "user", "content": user_input})
templated_input=tokenizer.apply_chat_template(
    chat_history,
    tokenize=False,
    add_generation_prompt=True
)
#分词转为张量
inputs=tokenizer(templated_input, return_tensors="pt").to(model.device)
outputs=model.generate(
    **inputs,
    max_new_tokens=max_new_tokens,
)
#解码输出
reply=tokenizer.decode(outputs[0], skip_special_tokens=True)
#提取Assistant的新回复部分
response=reply.split("assistant")[-1].replace(":", "").strip()
#存入历史记录
chat_history.append({"role": "assistant", "content": response})
return response

```

输入对话：

```

print("User: Hello, who are you?")
print("Assistant:", chat("Hello, who are you?"))

print("\nUser:What is the capital of China?")
print("Assistant:", chat("what is the capital of China?"))

print("\nUser:What local specialties are there here?")
print("Assistant:", chat("what local specialties are there here?"))

print("\nUser:What famous attractions are there here?")
print("Assistant:", chat("what famous attractions are there here?"))

```

输出结果如下，Qwen的输出格式带有思考过程，即<think>，同时也测试了其他问题，偶尔会出现客观上的错误，可能是模型小的原因，没有那么大的数据量来训练。

```

User: Hello, who are you?
Assistant: designed to help with questions and tasks. How can I assist you today? 😊

User:What is the capital of China?
Assistant: <think>
Okay, the user asked, "What is the capital of China?" I need to recall the correct answer. China's capital is Beijing. I should confirm that there's no confusion with other countries' capitals. Let me make sure there's no other city that's commonly known as the capital. Beijing is the only one I can think of. So, the response should be straightforward and accurate.
</think>

The capital of China is **Beijing**.

User:What local specialties are there here?
Assistant: <think>
Okay, the user just asked about local specialties in China. Let me think. I need to provide a helpful response. China has many cities and regions with unique cuisines. The user might be looking for food options that are popular in their area. Since they asked for local specialties, I should mention the main cities and their famous dishes.

First, I should list some major cities in China and their well-known dishes. Beijing is a big city, so mentioning Beijing's famous dishes like duck egg and dumplings would be good. Then, I can include other cities like Shanghai, where seafood is a big deal. It's important to note that the user might be in a city and wants to know about local food options, so providing a variety of cities and their specialties would be helpful.

I should also mention that the user might want to know more about specific dishes or regions. Including a friendly note at the end to ask if they need more information would be a good way to conclude the response. Need to make sure the information is accurate and helpful.
</think>

China has a rich culinary heritage with many local specialties across cities. Here are some popular ones
- **Beijing** Famous dishes like **duck egg and dumplings**, **soufu**, and

```

User: What famous attractions are there here?
Assistant: <think>
Okay, the user asked about famous attractions in China. Let me start by recalling the major cities and their famous landmarks. Beijing is a good start with the Summer Palace and the Great Wall. Then there's Shanghai with the Bund and the Shanghai Tower.

Next, I should mention other major cities like Guangzhou, Shenzhen, and Harbin. Each city has unique attractions. Guangzhou is known for its famous markets and the Golden Gate Bridge. Shenzhen is a tech hub with the Shanghai Tower and the Hongqiao Lake. Harbin has the Olympic Park and the Great Northern Mountains.

I need to make sure the answer is comprehensive but not too long. Also, check if the user wants more details on each attraction. Maybe add a note about cultural significance to give context. Avoid any errors in the names and information. Keep the tone friendly and helpful.

</think>

China has many iconic attractions across major cities

- **Beijing**: Summer Palace (Yuanmingyuan), Great Wall, and the Forbidden City.
- **Shanghai**: Bund, Shanghai Tower, and Hongqiao Lake.
- **Guangzhou**: Golden Gate Bridge and the famous Guangdong Market.
- **Shenzhen**: Shanghai Tower, Hongqiao Lake

2.2 分词

每个模型自带一个分词器tokenizer，可以将一句话分为各个带有id的tokens。以便进行后续的训练。

```

sentence="I like the class named GenAI,though i don't use transformers before."
tokens=tokenizer.tokenize(sentence)
token_ids=tokenizer.convert_tokens_to_ids(tokens)

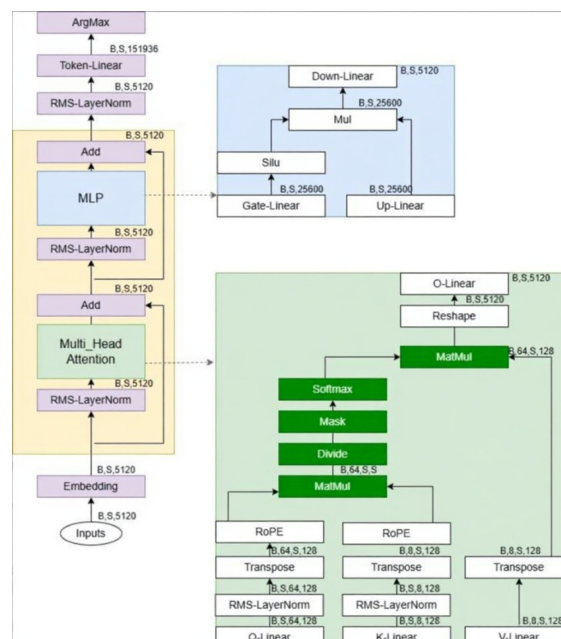
print("Tokens:", tokens)
print("Token IDs:", token_ids)

```

Tokens: ['I', 'Ġlike', 'Ġthe', 'Ġclass', 'Ġnamed', 'ĠGen', 'ĠAI', 'Ġ,', 'Ġ', 'Ġthough', 'Ġi', 'Ġdon', 'Ġ', 'Ġt', 'Ġuse', 'Ġtransformers', 'Ġbefore', 'Ġ.']
Token IDs: [40, 1075, 279, 536, 6941, 9316, 15469, 11, 4535, 600, 1513, 944, 990, 86870, 1573, 13]

2.3 Attention层输出

图为Qwen3-32B模型的结构，0.6B只是参数更小：



据Qwen3的模型文档以及代码，Qwen3共有28层，每层共有16个注意力头。

可在outputs时设置是否输出attention层，此处输出第0层的attention层的16个attention头的输出

```

inputs=tokenizer("What is the capital of China?", return_tensors="pt")
outputs=model(**inputs, output_attentions=True)

print(len(outputs.attentions))
print(outputs.attentions[0].shape) #(batch,heads,seq_len,seq_len)
print(outputs.attentions[0])

```

```

28
torch.Size([1, 16, 7, 7])

```

[illegible]